Assignment 3 Report: Equation Simplifier

CSCE625, Spring 2022

Name: David van Wijk UIN: 932001896

---

1 Solution Approach

(a) Algorithm At a high level, I am representing equations as nodes in a tree of possible valid manipulations of the original input equation, such that the equation simplification problem can be represented as a search problem. The algorithm I used to solve the formulated search problem was a modified Breadth-first search (BFS) algorithm. I intentionally selected BFS because of the completeness that it guarantees, meaning that it will always find a solution if indeed one exists. Additionally, I avoided using an algorithm such as Greedy Best-first search, because in the case of equation simplification, there may be times where to achieve a simpler final solution, we must first increase the complexity of the equation intentionally such as when completing the square. I was unable to incorporate capabilities of this nature, but my approach introduces the possibility of increased generality. The algorithm begins with the unsimplified equation inputted by the user, as the root of our tree of equations. Just like a normal BFS, my algorithm will terminate upon an empty frontier, and upon reaching the goal state. However the goal state is a bit trickier to define in this problem. If we construct the goal state such that it is too broad, we may exit the search with a simplified equation that could in fact be simplified further. For this reason, I decided to err on the side of caution, and made my goal state quite restrictive. Using a heuristic that will be described in paragraph c, I calculated a cost for each equation in the tree of equations, and if that cost is equal to 11, meaning we only have the variable of interest on the left hand side, and one node on the right hand side, we terminate the search. In all other cases however, we will continue the search.

The children are created using a number of valid possible actions or manipulations of an equation. Once the node is popped from the frontier, we will apply 8 possible actions, creating different manipulations of that node to push to the frontier. Just like in standard BFS, we will check these children one by one, see if they are in the list of explored nodes, and if not, we will add them to the first in first out (FIFO) queue. Additionally, to reduce the computation time, I am only allowing nodes to be added to the queue that don't have a *significantly* higher cost than others.

A main difference between a normal BFS algorithm, and this algorithm, is that upon popping the next node, we will check the cost of that node with our heuristic, and if that cost is less than the best cost we had previously stored, we will save this new node as our current best node, and update the best cost. We will repeat this process until we have reached the goal state, or until the queue of nodes is empty. If the loop exits due to an empty frontier, we will return the best node (i.e. equation) that we stored using the cost function. If, on the other hand, it exits due to reaching the goal state, we know we have the most simplified version of the input equation possible.

(b) Action Space As mentioned above, the children in our search space will be generated using 8 possible actions. The first five actions are purely simple equation manipulations, that have to do with <u>applying inverses</u>.

1. Left-Right swap: This swap is the simplest one. Here we swap the left side and the right side of the equation (e.g. if our equation was x = 5, this action would return 5 = x)

2. Left-Left node swap: This swap moves the left node of the left side of the equation over to the other side using inverses. (e.g. 3 + 5 = x $\longrightarrow$ 5 = x - 3)

3. Left-Right node swap: This swap moves the right node of the left side of the equation over to the other side using inverses. (e.g. 3 + 5 = x $\longrightarrow$ 3 = x - 5)

4. Right-Left node swap: This swap moves the left node of the right side of the equation over to the other side using inverses. (e.g. x = 3 + 5 $\longrightarrow$ x - 3 = 5)

5. Right-Right node swap: This swap moves the right node of the right side of the equation over to the other side using inverses. (e.g. x = 3 + 5 $\longrightarrow$ x - 5 = 3)

Additionally, there are internal checks within all of these functions that check if a) the nodes we are interested in switching do indeed exists, and b) the operator allows for a switch of this kind. If the action is not applicable, the original tree is returned (if interested, see code for more detail Appendix A).

The next valid action has to do with <u>arithmetic evaluation</u>. In the code, the function eval() performs this task for us. In eval(), we recursively examine the nodes of the inputted equation tree looking for situations in which we have adjacent children that are either of type INT or FLOAT. If we find such a case, we will use the operator that applies to both of these children, and combine them, creating a brand-new node with the arithmetic result of these

two values. As an aside, I made the choice to eventually convert all ints to floats, to avoid issues of applying int() to a float. Additionally, since I represented square roots as exponentials to the 1/2, this took care of potential issues there as well.

The next valid action addresses swaps associated with commutativity. The function in the code (Appendix A) called commutative() takes an equation tree as an input, and recursively examines the nodes of the tree, until it finds a plus or multiplication operator, and performs a swap on the children of this operator. It does this to the first instance of commutativity that it can find, but this isn't an issue, due to the fact that we will push this node to the queue and then eventually apply the whole range of actions on this new node again.

The last valid action that I considered was an action that performed swaps according to associativity rules. Associativity applies to multiplication and to addition, shown in Equation 1 and Equation 2

$$a + (b + c) = (a + b) + c \tag{1}$$

$$a * (b * c) = (a * b) * c \tag{2}$$

In the case of my code, I again constructed a recursive function that takes an equation tree as an input, and finds the first instance of a case where we can apply an associative rule, on either side of the equation, and performs that swap. In my case, I constructed this as two separate checks on each side of the equation. On each side of the equation, I checked to see if there was a + or * operator followed by another + or * operator on either the left or right side of the equation. Upon finding either one of these situations, we apply the associative rule by swapping the appropriate nodes (please see Appendix A for more detail).

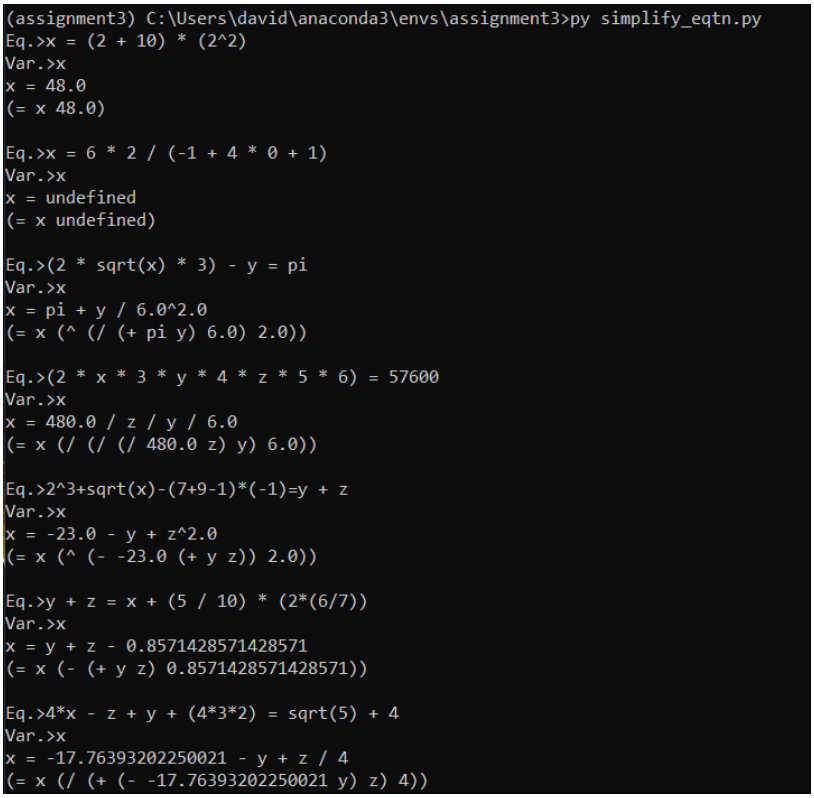
(c) Heuristic The heuristic is a cost function that I use to rank the simplicity and "goodness" of an equation tree. In the code, this function called costFunction() takes in an equation tree, and the string representing the variable of interest. I experimented with a number of different cost functions, for example, I used one that used a Depth-first search (DFS) to return the depth at which our variable was, however, since the cost function is being called every time we generate a new set of children, I spent quite a bit of time working on a cost function that would perform calculations quickly and was as simple as possible. The current cost function calculates the cost of the inputted equation based on only 3 parameters:

1. Cost for the variable being on the right hand side of the equation (c_1)
2. Cost for nodes being on the same side of the equation the variable is on (c_2)
3. Cost for nodes being on the opposite side of the variable (c_3)

Clearly, the costs of these three parameters must be tuned carefully - we cannot give c_2 the same weight as c_3, otherwise we wouldn't "incentivize" the equations to move variables to the opposite side as the variable of interest. After some experimentation, I went with a weight of $c_1 = 20$, $c_2 = 10$, $c_3 = 1$. The purpose of c_1 is to get our equations always in the form of x = [...], and the purpose of c_3 is to reduce the number of total nodes whenever possible.

2 Examples of Code

In Figure 1, we can see a few examples of the code being run on input equations to try to demonstrate the effectiveness of the algorithm. There are a few items of note. The first is that in the fourth example, the equation does not simplify fully, which is due to restrictions that I imposed on the depth of the tree of equations, thus imposing a restriction on run time. Due to the very large number of possible actions (and thus children at each level), and the curse of dimensionality, there is a very large number of combinations of all the possible actions described in section 1. When I think about the utility of an equation simplifier, if you have to wait longer than a few seconds, it might be simpler to just do the simplification by hand which is why I imposed this restriction. If I loosen the restriction I am able to produce the fully simplified version of the equation, but in a much longer time frame. Secondly, there are cases in which we clearly have floats even though ints were inputted. As described in section 1 I eventually convert all ints to floats in order to avoid computing an int of a float, and thus avoid losing information. It also is more compatible with the way I addressed square roots.



```
(assignment3) C:\Users\david\anaconda3\envs\assignment3>py simplify_eqtn.py
Eq.>x = (2 + 10) * (2^2)
Var.>x
x = 48.0
(= x 48.0)

Eq.>x = 6 * 2 / (-1 + 4 * 0 + 1)
Var.>x
x = undefined
(= x undefined)

Eq.>(2 * sqrt(x) * 3) - y = pi
Var.>x
x = pi + y / 6.0^2.0
(= x (^ (/ (+ pi y) 6.0) 2.0))

Eq.>(2 * x * 3 * y * 4 * z * 5 * 6) = 57600
Var.>x
x = 480.0 / z / y / 6.0
(= x (/ (/ (/ 480.0 z) y) 6.0))

Eq.>2^3+sqrt(x)-(7+9-1)*(-1)=y + z
Var.>x
x = -23.0 - y + z^2.0
(= x (^ (- -23.0 (+ y z)) 2.0))

Eq.>y + z = x + (5 / 10) * (2*(6/7))
Var.>x
x = y + z - 0.8571428571428571
(= x (- (+ y z) 0.8571428571428571))

Eq.>4*x - z + y + (4*3*2) = sqrt(5) + 4
Var.>x
x = -17.76393202250021 - y + z / 4
(= x (/ (+ (- -17.76393202250021 y) z) 4))
```

Figure 1: Code being run for a few examples

3 Running the Code

Running the code is very straightforward. If one has the modified version of the equation parser, and util.py from the Berkeley AI Pacman assignment, as well as Python 3, the user must simply run simplify_eqtn.py. There will then be a prompt to enter an equation, as well as the variable of interest, and upon pressing enter, the algorithm will return the solution. This entire process will run indefinitely, or until the user decides they're better off using Mathematica.

4 General Notes

All of the classes and functions that I created to run the search algorithm are contained in simplify_eqtn.py, and all the names of the functions and classes should be self-evident. In the validMovements class, I have all the 5 possible swapping actions that can be performed, and beneath that class I have all the recursive functions. The swapping functions are represented as lambda functions to reduce the complexity of the code. I modified the equation parser in p_expression_single_fun() to convert square roots to an exponential (i.e. the original leaf to the 1/2). I also modified p_expression_uminus() to handle a negative node as negative of that node rather than a unary operation.

5 Limitations

This equation solver solution is only partially complete. It is unable to perform calculus operations, and is unable to perform simplifications on unary functions such as log, sin etc. Additionally, it is not able to utilize identities to make simplifications. However, given more time I think I would have been able to deal with identities in a very similar way as with the eval() function, by simply recursively looking through the tree and finding identities. However, this method would require coding certain identities individually and would be quite tedious. To my knowledge there should be no bugs if the equations stay within the restrictions described above. Additionally, there is a very large number of possible actions, which causes quite a bloated frontier. If I were to redo my approach, I would focus on creating a smaller number of more general actions.

6 Acknowledgments

For this assignment I used the starter code provided by Professor Shell, and I used the util.py package from the Berkeley AI Pacman assignment `http://ai.berkeley.edu/search.html` since I based my search algorithms on the ones I wrote for that assignment. I also worked with Matthew Kocmoud and Moeez Akmal to discuss high-level concepts and methods to approach this problem.

A Python Code

A.1 simplify_eqtn.py

```python
# David van Wijk, davidvanwijk@tamu.edu
#
# CSCE625: Artificial Intelligence
# Programming Assignment 3 : Simplifying Mathematical Expressions via Search

import eqparser
import copy
import util

class validMovements:

    # Swaps the left and right nodes of the tree
    def L_R_swap(tree):
        working_tree = copy.deepcopy(tree)
        working_tree.children[0] = tree.children[1]
        working_tree.children[1] = tree.children[0]
        return working_tree

    # Move the left-left node over
    # Also checks for if left-left node even exists
    # if not, returns tree w/o modification
    def LL_node_swap(tree):
        working_tree = copy.deepcopy(tree)
        node_list = str(repr(tree.children[0])).split()
        num_nodes = len(node_list)
        if num_nodes != 1:
            R = tree.children[1]
            LL = tree.children[0].children[0]
            LR = tree.children[0].children[1]
            L_op = tree.children[0].leaf
            if L_op == '^' or L_op == '/':
                return tree
            else:
                op = oppositeOp(L_op)
                newRHS = eqparser.Node('BINARYOP', [R, LL], op)
            working_tree.children[0] = LR
            working_tree.children[1] = newRHS
        return working_tree

    # Move the left-right node over
    def LR_node_swap(tree):
        working_tree = copy.deepcopy(tree)
        node_list = str(repr(tree.children[0])).split()
        num_nodes = len(node_list)
        if num_nodes != 1:
            R = tree.children[1]
            LL = tree.children[0].children[0]
            LR = tree.children[0].children[1]
            L_op = tree.children[0].leaf
            if L_op == '^':
                newLR = eqparser.Node('FLOAT', [], 1/float(str(LR)))
                newRHS = eqparser.Node('BINARYOP', [R, newLR], L_op)
            else:
                op = oppositeOp(L_op)
                newRHS = eqparser.Node('BINARYOP', [R, LR], op)
            working_tree.children[0] = LL
            working_tree.children[1] = newRHS
        return working_tree

    # Move the right-left node over
    def RL_node_swap(tree):
        working_tree = copy.deepcopy(tree)
```

```python
        node_list = str(repr(tree.children[1])).split()
        num_nodes = len(node_list)
        if num_nodes != 1:
            L = tree.children[0]
            RL = tree.children[1].children[0]
            RR = tree.children[1].children[1]
            R_op = tree.children[1].leaf
            if R_op == '^' or R_op == '/':
                return tree
            else:
                op = oppositeOp(R_op)
                newLHS = eqparser.Node('BINARYOP', [L, RL], op)
            working_tree.children[1] = RR
            working_tree.children[0] = newLHS
        return working_tree

    # Move the right-right node over
    def RR_node_swap(tree):
        working_tree = copy.deepcopy(tree)
        node_list = str(repr(tree.children[1])).split()
        num_nodes = len(node_list)
        if num_nodes != 1:
            L = tree.children[0]
            RL = tree.children[1].children[0]
            RR = tree.children[1].children[1]
            R_op = tree.children[1].leaf
            if R_op == '^':
                newRR = eqparser.Node('FLOAT', [], 1/float(str(RR)))
                newLHS = eqparser.Node('BINARYOP', [L, newRR], R_op)
            else:
                op = oppositeOp(R_op)
                newLHS = eqparser.Node('BINARYOP', [L, RR], op)
            working_tree.children[1] = RL
            working_tree.children[0] = newLHS
        return working_tree

# Using recursion, check down the tree until we find something to combine
def eval(tree):

    if len(str(repr(tree.children[0])).split()) != 1:

        if  (tree.children[0].children[0].type == 'INT' or
            tree.children[0].children[0].type == 'FLOAT') and \
            (tree.children[0].children[1].type == 'INT' or
            tree.children[0].children[1].type == 'FLOAT'):
                operator = tree.children[0].leaf
                if operator == '^':
                    value = (float(str(tree.children[0].children[0]))) ** \
                        (float(str(tree.children[0].children[1])))
                    tree.children[0] = eqparser.Node('FLOAT', [], value)
                if operator == '+':
                    value = (float(str(tree.children[0].children[0]))) + \
                        (float(str(tree.children[0].children[1])))
                    tree.children[0] = eqparser.Node('FLOAT', [], value)
                if operator == '*':
                    value = (float(str(tree.children[0].children[0]))) * \
                        (float(str(tree.children[0].children[1])))
                    tree.children[0] = eqparser.Node('FLOAT', [], value)
                if operator == '/':
                    value = (float(str(tree.children[0].children[0]))) / \
                        (float(str(tree.children[0].children[1])))
                    tree.children[0] = eqparser.Node('FLOAT', [], value)
                if operator == '-':
                    value = (float(str(tree.children[0].children[0]))) - \
                        (float(str(tree.children[0].children[1])))
                    tree.children[0] = eqparser.Node('FLOAT', [], value)
```

```python

        else:
            # Check not satisfied, go deeper
            eval(tree.children[0])

    if len(str(repr(tree.children[1])).split()) != 1:
        if (tree.children[1].children[0].type == 'INT' or
           tree.children[1].children[0].type == 'FLOAT') and \
           (tree.children[1].children[1].type == 'INT' or
            tree.children[1].children[1].type == 'FLOAT'):
                operator = tree.children[1].leaf
                if operator == '^':
                    value = (float(str(tree.children[1].children[0]))) ** \
                        (float(str(tree.children[1].children[1])))
                    tree.children[1] = eqparser.Node('FLOAT', [], value)
                if operator == '+':
                    value = (float(str(tree.children[1].children[0]))) + \
                        (float(str(tree.children[1].children[1])))
                    tree.children[1] = eqparser.Node('FLOAT', [], value)
                if operator == '*':
                    value = (float(str(tree.children[1].children[0]))) * \
                        (float(str(tree.children[1].children[1])))
                    tree.children[1] = eqparser.Node('FLOAT', [], value)
                if operator == '/':
                    value = (float(str(tree.children[1].children[0]))) / \
                        (float(str(tree.children[1].children[1])))
                    tree.children[1] = eqparser.Node('FLOAT', [], value)
                if operator == '-':
                    value = (float(str(tree.children[1].children[0]))) - \
                        (float(str(tree.children[1].children[1])))
                    tree.children[1] = eqparser.Node('FLOAT', [], value)

        else:
            # Check not satisfied, go deeper
            eval(tree.children[1])

# Commutivity using recursion
# Finds a node to swap (for + or * and performs the first swap it can find)
def commutative(tree):
    if len(str(repr(tree.children[0])).split()) != 1:

        operator = tree.children[0].leaf
        R = copy.deepcopy(tree.children[0].children[1])
        L = copy.deepcopy(tree.children[0].children[0])
        if operator == '+' or operator == '*':
            tree.children[0].children[0] = R
            tree.children[0].children[1] = L
        else:
            # if not, go deeper
            commutative(tree.children[0])

    if len(str(repr(tree.children[1])).split()) != 1:

        operator = tree.children[1].leaf
        R = copy.deepcopy(tree.children[1].children[1])
        L = copy.deepcopy(tree.children[1].children[0])

        if operator == '+' or operator == '*':
            tree.children[1].children[0] = R
            tree.children[1].children[1] = L
        else:
            # if not, go deeper
            commutative(tree.children[1])

# Associativity using recursion
# Finds nodes to swap (for + or * and performs the first swap it can find)
```

```python
# If we have * or + as the parent to something and another * or + we can perform a swap
def associative(tree):

    # LEFT
    if len(str(repr(tree.children[0])).split()) > 4:
        operator_top = tree.children[0].leaf

        # Right side operator
        if len(str(repr(tree.children[0].children[1])).split()) > 2:
            operator_lower_R = tree.children[0].children[1].leaf
            L = copy.deepcopy(tree.children[0].children[0])
            # RR = copy.deepcopy(tree.children[0].children[1].children[1])
            RL = copy.deepcopy(tree.children[0].children[1].children[0])
            if (operator_top == '+' and operator_lower_R == '+') or \
                (operator_top == '*' and operator_lower_R == '*'):
                tree.children[0].children[0] = RL
                tree.children[0].children[1].children[0] = L

        # Left side operator
        if len(str(repr(tree.children[0].children[0])).split()) > 2:
            operator_lower_L = tree.children[0].children[0].leaf
            R = copy.deepcopy(tree.children[0].children[1])
            LR = copy.deepcopy(tree.children[0].children[0].children[1])
            # LL = copy.deepcopy(tree.children[0].children[0].children[0])
            if (operator_top == '+' and operator_lower_L == '+') or \
                (operator_top == '*' and operator_lower_L == '*'):
                tree.children[0].children[1] = LR
                tree.children[0].children[0].children[1] = R
        else:
            # if not, go deeper
            associative(tree.children[0])

    # RIGHT
    if len(str(repr(tree.children[1])).split()) > 4:
        operator_top = tree.children[1].leaf

        # Right side operator
        if len(str(repr(tree.children[1].children[1])).split()) > 2:
            operator_lower_R = tree.children[1].children[1].leaf
            L = copy.deepcopy(tree.children[1].children[0])
            # RR = copy.deepcopy(tree.children[1].children[1].children[1])
            RL = copy.deepcopy(tree.children[1].children[1].children[0])
            if (operator_top == '+' and operator_lower_R == '+') or \
                (operator_top == '*' and operator_lower_R == '*'):
                tree.children[1].children[0] = RL
                tree.children[1].children[1].children[0] = L

        # Left side operator
        if len(str(repr(tree.children[1].children[0])).split()) > 2:
            operator_lower_L = tree.children[1].children[0].leaf
            R = copy.deepcopy(tree.children[1].children[1])
            LR = copy.deepcopy(tree.children[1].children[0].children[1])
            # LL = copy.deepcopy(tree.children[1].children[0].children[0])
            if (operator_top == '+' and operator_lower_L == '+') or \
                (operator_top == '*' and operator_lower_L == '*'):
                tree.children[1].children[1] = LR
                tree.children[1].children[0].children[1] = R
        else:
            # if not, go deeper
            associative(tree.children[1])

def oppositeOp(string):
    if string == '^':
        pass
    elif string == '*':
        oppOp = '/'
```

```python
    elif string == '/':
        oppOp = '*'
    elif string == '+':
        oppOp = '-'
    elif string == '-':
        oppOp = '+'
    return oppOp

# Returns an integer where higher numbers are higher priority for operations
def assignPrecedence(string):
    if string == '^':
        priority = 3
    elif string == '*' or string == '/':
        priority = 2
    elif string == '+' or string == '-':
        priority = 1
    return priority

# Should take an equation tree as an input, score the tree, and return the score
# 1) Cost for variable being on RHS
# 2) Cost for nodes on side of var besides the var itself
# 3) Cost for nodes being on the opposite var side
def costFunction(tree,var):
    # Tunable costs:
    RHS_cost = 20
    Nodes_on_var_side = 10
    Nodes_on_non_var_side = 1
    cost = 0
    if var in str(tree.children[1]):
        cost = cost + RHS_cost

    for i in range(2):
        if var in str(tree.children[i]):
            node_list = str(repr(tree.children[i])).split()
            num_nodes = len(node_list)
            cost = cost + Nodes_on_var_side*num_nodes
        else:
            node_list = str(repr(tree.children[i])).split()
            num_nodes = len(node_list)
            cost = cost + Nodes_on_non_var_side*num_nodes

    return cost

def breadthFirstSearch(tree,var):
    # Uses a FIFO queue for the frontier meaning new nodes go to the back of the queue
    # and the old nodes which are shallower than new nodes get expanded first
    # Use the Queue function provided in util (from pacman project)

    # Following pseudocode from: https://en.wikipedia.org/wiki/Breadth-first_search

    # All selected actions that can be done to equation trees
    actions = [lambda x: validMovements.L_R_swap(x),
               lambda x: validMovements.LL_node_swap(x),
               lambda x: validMovements.LR_node_swap(x),
               lambda x: validMovements.RR_node_swap(x),
               lambda x: validMovements.RL_node_swap(x)]

    best_cost = 1e6
    maxDepth = 100
    best_node = tree

    c0 = costFunction(tree,var)
    root = {'tree': tree, 'cost': [c0], 'depth': 0}
    explored = [repr(root['tree'])]
    frontier = util.Queue()
    frontier.push(root)
```

```python
    try:
        while not frontier.isEmpty():
            node = frontier.pop()
            if node['cost'][0] < best_cost:
                best_cost = node['cost'][0]
                best_node = node
                # print(node['tree'])
            if (node['depth'] > maxDepth) or node['cost'][0] == 11:
                return best_node
            else:
                # Apply action to current node for all possible actions
                for action in actions:
                    child = action(node['tree'])
                    child_evaled = copy.deepcopy(child)
                    child_commutative = copy.deepcopy(child)
                    child_associative = copy.deepcopy(child)
                    eval(child_evaled)
                    commutative(child_commutative)
                    associative(child_associative)
                    if repr(child) not in explored:
                        cost_node = costFunction(child,var)
                        explored.append(repr(child))
                        newNode = {'tree': child, 'cost': [cost_node],
                         'depth': node['depth']+1}
                        if cost_node < best_cost*1.2:
                            frontier.push(newNode)
                    if repr(child_evaled) not in explored:
                        cost_node = costFunction(child_evaled,var)
                        explored.append(repr(child_evaled))
                        newNode2 = {'tree': child_evaled, 'cost': [cost_node],
                         'depth': node['depth']+1}
                        if cost_node < best_cost*1.2:
                            frontier.push(newNode2)
                    if repr(child_commutative) not in explored:
                        cost_node = costFunction(child_commutative,var)
                        explored.append(repr(child_commutative))
                        newNode3 = {'tree': child_commutative, 'cost': [cost_node],
                         'depth': node['depth']+1}
                        if cost_node < best_cost*1.2:
                            frontier.push(newNode3)
                    if repr(child_associative) not in explored:
                        cost_node = costFunction(child_associative,var)
                        explored.append(repr(child_associative))
                        newNode4 = {'tree': child_associative, 'cost': [cost_node],
                         'depth': node['depth']+1}
                        if cost_node < best_cost*1.2:
                            frontier.push(newNode4)

        return best_node
    except ZeroDivisionError:
        s = var + ' = undefined'
        node['tree'] = eqparser.parse(s)
        return node

def depthFirstSearch(tree, var):
    # Following pseudocode from: https://en.wikipedia.org/wiki/Depth-first_search
    # Searches for var within the tree, returns:
    #  (1) the depth var found at
    #  (2) bool_left representing 0 or 1 based on side of equation var is on

    root = {'state': [tree], 'parent': [], 'depth': 0}
    explored = []
    frontier = []
    frontier.append(root)
    while frontier:
        node = frontier.pop()
```

```
        if (var == str(node['state'][0])) and (len(str(node['state'][0])) == len(var)):
            # BACKTRACK TO GET THE PARENTS OF X UNTIL WE GET BACK TO THE START
            backtrack_node = node
            while backtrack_node['depth'] != 1:
                for k in range(len(explored)):
                    if backtrack_node['parent'] == explored[k]['state']:
                        backtrack_node = explored[k]
                        break

            if backtrack_node['state'][0] == tree.children[0]:
                # print('left hand side')
                bool_left = 1
            else:
                #print('right hand side')
                bool_left = 0

            return node['depth'], bool_left
        else:
            if str(node['state'][0]) not in explored:
                explored.append(node)
                # explored.append(node['state'][0])
                children = node['state'][0].children
                for i in range(len(children)):
                    child = children[i]
                    newNode = {'state': [child], 'parent': node['state'],
                     'depth': node['depth']+1}
                    frontier.append(newNode)

def main_search():
    inputString = input('Eq.>')
    var = input('Var.>')
    inputEquation = eqparser.parse(inputString)
    finalTree = breadthFirstSearch(inputEquation,var)
    print(finalTree['tree'])
    print(repr(finalTree['tree']))
    print('')

while True:
    main_search()
```